Tipos Abstratos de Dados (TAD)

Estruturas de Dados e Introdução à Projeto e Análise de Algoritmos

Prof. Luis Souza

Tipos Abstratos de Dados (TAD)

- Na linguagem Python, os tipos de dados são dinâmicos, estabelecidos em tempo de execução. Este comportamento já proporciona a reutilização de variáveis para a utilização em diversos problemas (mas outros podem surgir).

- De linguagens clássicas e mais engessadas de implementação (como C), existe o conceito de tipo abstrato de dados, onde são definidos objetos específicos e operações que manipulam estes objetos. É de responsabilidade do desenvolvedor implementar os objetos e os métodos que operam sobre estes objetos, ficando para o usuário somente sua utilização (pela interface).

- A interface define o comportamento das operações implementadas para o objeto.

TAD

- A estrutura básica de um TAD é, portanto, a seguinte:
  - O tipo dos dados armazenados (conceitual, int, bool, list, etc)
  - As operações suportadas sobre estes dados
  - Os parâmetros das operações
  - O que cada operação faz (não como faz)
- Em Python, os TADs são “representados” pelas classes. Os atributos de uma classe definem os objetos (variáveis, tipos e estado) e os métodos definem sua manipulação (operações e parâmetros).
- Chamamos de TAD opaco aquele que implementa restrições de operação e acesso sobre seus objetos somente por métodos do próprio TAD (difícil torná-lo totalmente opaco no Python).

TAD

- Ao escrever um programa orientado a objetos: pensar no tipo de dado e nas operações sobre esses dados (isolando o conceito do particular problema)
  - Dizemos então que estamos criando uma abstração desse tipo de dado.
  - Com isso, podemos construir um tipo de dado que seja reutilizável para resolver também outros problemas similares com o mínimo de adaptações para ser estendido.
- Python já é construído para ser orientado a objeto: o tipo de dado do Python é dinâmico, conhecido e estabelecido em tempo de execução. O mesmo nome, ou referência pode no mesmo programa estar associado a objetos de diferentes tipos.

TAD

- Podemos ter:

```
x = 1
...
x = 'exemplo'
...
x = True
...
x = [4, 6, “chatice”, True, (2,5,4)]
…
```

TAD

- Podemos ter:

```
x = 1
...
x = 'exemplo'
...
x = True
...
x = [4, 6, “chatice”, True, (2,5,4)]
…
```

Atribuições de tipos diferentes para a mesma variável - variáveis são dinâmicas!

TAD

- Funções podem, portanto, serem construídas e funcionarem para diversos tipos diferentes de dados em Python, sem a necessidade de adaptações.

```
def Max_Valor(seq):

    '''
    Recebe uma sequência (lista, tupla, string, etc.)
    Devolve o máximo
    '''

    mv = seq[0]
    for k in range(1, len(seq)):
        if mv < seq[k]: mv = seq[k]
    return mv
```

TAD

- Funções podem, portanto, serem construídas e funcionarem para diversos tipos diferentes de dados em Python, sem a necessidade de adaptações.

Qualquer objeto iterável pode ser utilizado como parâmetro!

```
def Max_Valor(seq):

    '''
    Recebe uma sequência (lista, tupla, string, etc.)
    Devolve o máximo
    '''

    mv = seq[0]
    for k in range(1, len(seq)):
        if mv < seq[k]: mv = seq[k]
    return mv
```

TAD

Testes:

```
# testes de Max_Valor
# com lista de int
x = [5, 2, 8, 0]
print("lista - máximo valor de ", x, " = ", Max_Valor(x))
# com tupla de int
x = (5, 2, 8, 0)
print("tupla - máximo valor de ", x, " = ", Max_Valor(x))
# com string
x = "abracadabra"
print("string - máximo valor de ", x, " = ", Max_Valor(x))
# com lista de string
x = ["cinco", "dois", "oito", "zero"]
print("lista de string - máximo valor de ", x, " = ", Max_Valor(x))
# com lista de tuplas
x = [(3, 4, 2), (3, 4, 6), (3, 4, 4)]
print("lista de tuplas - máximo valor de ", x, " = ", Max_Valor(x))
# com tupla de tuplas
x = ((3, 4, 2), (3, 4, 6), (3, 4, 4))
print("tupla de tuplas - máximo valor de ", x, " = ", Max_Valor(x))
```

TAD

- A independência do tipo dos dados possibilita interfaces para o usuário de modo que a relação entre os algoritmos e as estruturas de dados não sejam explícitas (isso quebra o conceito de TAD, infelizmente).
- Rigorosamente, na função *Max_Valor* teríamos que garantir a consistência do tipo. O tipo dos elementos da sequência *seq* deve sempre ser o mesmo.

```
def Max_Valor(seq):
'''
Recebe uma sequência (lista, tupla, string, etc.).
Devolve o máximo desta.
'''
mv, tmv = seq[0], type(seq[0])
for k in range(1, len(seq)):
       # consistência dos tipos
       if type(seq[k]) != tmv:
           raise ValueError("tipos diferentes")
           if mv < seq[k]: mv = seq[k]
    return mv
```

Definição de Tipos Abstratos de Dados (TAD)

- TAD: conjunto de objetos e as operações sobre esses objetos.
  - As operações definem uma interface entre o TAD e o resto do programa.
  - A interface define o comportamento destas operações (o que elas fazem e não como fazem).
  - Para o usuário final: abstração que isola o resto do programa das estruturas de dados e algoritmos utilizados (o usuário não está interessado na implementação, mas em como utilizá-la).
- O TAD é um “modelo” de estrutura de dados que especifica:
  - O tipo de dados armazenado (tipo conceitual, não int, bool, list, etc.).
  - As operações suportadas sobre esses dados (+ os parâmetros destas operações).
  - O que cada operação faz e não como faz (interface).

TAD em Python

- Em Python, a representação de um TAD é feita por meio das classes.
  - As classes com seus atributos (as variáveis que representam os dados, os seus tipos e que definem o estado deste objeto).
  - Os métodos (as operações, parâmetros e o que pode ser feito sobre um objeto desta classe), implementam esses conceitos.
- **Toda a construção de TAD deve ser feita utilizando o conceito de abstração e encapsulamento proporcionado pelas classes!!!**
  - A linguagem Python já é toda planejada como orientada a objetos (classes).
  - Não é possível o encapsulamento total em Python -> nem com *name mangling*

Exemplo de TAD - Fração Ordinária (classe Fracao)

- Fração ordinária definida por dois inteiros:
  - a: numerador
  - b: denominador (>0)
    - o numerador é negativo quando a fração é negativa
- Classe Fracao:
  - construtor da classe: __init__
  - método somaFracao(f1, f2): devolve f1+f2
  - método mostraFracao(f1): mostra f1 no formado a/b

Exemplo de TAD

- Uma fração ordinária é definida por dois inteiros a e b onde a é o numerador e b, o denominador (a/b), com b > 0. Consideramos que o numerador é negativo quando a fração é negativa.

```
class Fracao:
    '''Define a classe Fracao '''
    # Construtor da classe
    def __init__(self, topo = 0, base = 1):
        '''Cria uma instancia de Fracao .'''
        self.num = topo
        self.den = base
    # Retorna a soma de dois elementos da classe
    def somaFracao(self, f1, f2):
        self.num = f1.num * f2.den + f1.den * f2.num
        self.den = f1.den * f2.den
    # Mostre uma fração na forma a/b
    def mostraFracao(self):
        print(self.num, "/", self.den)
```

Exemplo de TAD

- Uma fração ordinária é definida por dois inteiros a e b onde a é o numerador e b, o denominador (a/b), com b > 0. Consideramos que o numerador é negativo quando a fração é negativa.

A classe define o TAD

Definição do construtor da classe (objeto)

Definição dos métodos do TAD

```
class Fracao:
    '''Define a classe Fracao'''
    # Construtor da classe
    def __init__(self, topo = 0, base = 1):
        '''Cria uma instancia de Fracao.'''
        self.num = topo
        self.den = base
    # Retorna a soma de dois elementos da classe
    def somaFracao(self, f1, f2):
        self.num = f1.num * f2.den + f1.den * f2.num
        self.den = f1.den * f2.den
    # Mostre uma fração na forma a/b
    def mostraFracao(self):
        print(self.num, "/", self.den)
```

Exemplo de TAD

- Uma fração ordinária é definida por dois inteiros a e b onde a é o numerador e b, o denominador (a/b), com b > 0. Consideramos que o numerador é negativo quando a fração é negativa.

```
# Exemplos de uso da classe Fração

x = Fracao(3, 4)
y = Fracao(5, 12)
z = Fracao()

# z = x + y
z.somaFracao(x, y)
z.mostraFracao()

# z = 4/5 + 3/7
z.somaFracao(Fracao(4, 5), Fracao(3, 7))
z.mostraFracao()
```

Exemplo de TAD

- Uma fração ordinária é definida por dois inteiros a e b onde a é o numerador e b, o denominador (a/b), com b > 0. Consideramos que o numerador é negativo quando a fração é negativa.

```
# Exemplos de uso da classe Fração

x = Fracao(3, 4)
y = Fracao(5, 12)
z = Fracao()

# z = x + y
z.somaFracao(x, y)
z.mostraFracao()

# z = 4/5 + 3/7
z.somaFracao(Fracao(4, 5), Fracao(3, 7))
z.mostraFracao()
```

Definição das instâncias do objeto (TAD)

Operações sobre os objetos criados

Operações sobre os objetos criados

Exemplo de TAD

Algumas observações sobre a solução acima:

1. Seria conveniente guardar o resultado da soma na forma mais simples. 56/48 é o mesmo que 7/6. Para tanto, basta dividir o numerador e denominador pelo máximo divisor comum entre eles. Vamos então incorporar a função mdc(n, m) dentro da nossa classe.
2. Seria também conveniente, em vez de usar a função somaFracao(x,y) usar o mecanismo de sobrecarga de operadores e escrever **`z = x + y`** em vez de **`z.somaFracao(x, y)`**. O programa fica muito mais claro. Para isso a função de somar passará a ser **__add__(self,...)**.
3. Seria também conveniente, usar o mesmo mecanismo para escrever **print(z)** em vez de z.mostraFuncao(). Para isso, teremos que transformar uma fração em uma string, construindo a função **__str__(self,...)**. Importante entender o mecanismo de sobreposição de métodos. A função print sempre tenta transformar os seus parâmetros em string. Quando o parâmetro pertence a uma determinada classe, a função __str__() desta classe é chamada. Se existe, a conversão é feita e o print segue normalmente. Se não existe a conversão para esta classe é impresso um erro.
4. O denominador de uma Fração não pode ser zero. Assim vamos sinalizar essa exceção.

Solução

Exercícios

- Complete a classe acima, definindo as operações de subtração, multiplicação e divisão de frações (operadores -, * e /).

- Refaça os operadores maior, menor, maior ou igual, menor ou igual, e diferente (funções *__gt__, __lt__, __ge__, __le__, __ne__*).

TAD Conjunto (utilizando listas)

- Supondo que a classe set exista, ou mesmo que para uma aplicação específica queremos representar um conjunto como uma lista: vamos implementar a nossa própria classe Conjunto -> estará classificado em ordem crescente dos elementos.

- O construtor da classe: receberá uma lista, eliminará repetições de elementos e ordenará a lista.

TAD Conjunto

- Função que elimina repetições de elementos de uma lista:

```
def Elimina_Repetidos(lista):
    nova_lista = []
    for k in range(0, len(lista)):
        tem_rep = False
        # procura lista[k] nos elementos à frente
        for j in range(k + 1, len(lista)):
            if lista[k] == lista[j]:
                tem_rep = True
                break
        # se não tem repetidos inclua na nova lista
        if not tem_rep: nova_lista.append(lista[k])
    return nova_lista
```

TAD Conjunto

```
class Conjunto:
    ''' Define a classe Conjunto'''
    # Construtor da classe
    def __init__(self, lista = []):
        ''' Cria uma instância de Conjunto '''
        self.lset = Elimina_Repetidos(lista)
        self.lset.sort()
    # união (+)
    def __add__(s1, s2):
        # cria Conjunto com a concatenação de duas listas
        return Conjunto(s1.lset + s2.lset)
    # Tamanho do Conjunto
    def __len__(self):
        return len(self.lset)
    # Transforma elemento da classe em str para o print
    # Mostrar um '*' na frente para diferenciar do print(lista)
    def __str__(self):
        return '*' + str(self.lset)
```

Testes TAD Conjunto

```
x = Conjunto([1, 2, 25, 5, 72, 5, 1])
print(len(x))
print(x)

y = Conjunto([])
print(len(y))
print(y)

z = Conjunto([1, 9, 113, 5, 9, 2, 22, 12, 2, 11])
print(len(z))
print(z)

t = x + z
print(t)
print(x + z)
```

TAD Conjunto

Exercícios:

3. Existem formas mais simples de escrever a Elimina_Repetidos acima. Escreva-a de outras maneiras.
4. Incremente agora a classe Conjunto acima definindo as operações de intersecção de conjuntos com o operador * (__mul__) e subtração de conjuntos com o operador – (__sub__).
5. Idem com as operações booleanas “está contido” com o operador < (__lt__), “contém” com o operador > (__gt__) e “igual” com o operador == (__eq__).,
6. Idem com a operação “pertinência” com o operador in (__contains__), “inclusão” de elemento com += (__iadd__) e exclusão com -= (__isub__).

TAD Polinomio (com listas)

- Um polinômio pode ser representado como uma lista de valores float, onde o elemento j representa o coeficiente de x^j . Portanto uma lista com n floats representa um polinômio de grau n-1.

```
class Polinomio:
    ''' Define a classe Polinomio'''
    def __init__(s, coef):
        s.cfs = coef[:] # coeficientes
    # Devolve o valor do polinômio no ponto x
    def valor(self, x):
        soma = 0
        for k in range(len(self.cfs)):
            soma += self.cfs[k] * pow(x, k)
        return soma
```

```
# testes
p1 = Polinomio([1.0, -2.2, 3.5])
v = p1.valor(1.0)
print(v)
```

TAD Polinomio

Exercício:

7. Implemente as operações usuais: soma, subtração, multiplicação, divisão, derivação e integração para a classe Polinômio. Algumas das operações alteram o grau do polinômio.

TAD Matriz

```
# testes
ma = Matriz(2, 3)
mb = Matriz(2, 3)
# define as linhas
ma[0] = [1, 2, 3]
ma[1] = [4, 5, 6]
mb[0] = [5, 5, 5]
mb[1] = [9, 9, 9]
# some
mc = ma + mb
# imprima
```

Exercício:

8. Implemente uma classe Matriz, baseada no exemplo de uso acima. O construtor da classe Matriz(self, m, n) cria uma lista de listas de m por n elementos zerados. Implemente as operações de subtração de matrizes (-) e multiplicação (*). Importante verificar a compatibilidade do tamanho das listas antes de realizar as operações.



Obrigado!

Contato:

- la.souza@inf.ufes.br
- github.com/luisouza
- https://scholar.google.com.br/citations?user=3gMIEPYAAAAJ&hl=pt-BR
- https://www.researchgate.net/profile/Luis-Souza-Jr